

Conceptual Data Mapping: IDTA 02035-1 Digital Battery Passport (Part 1) ↔ OpenUSD (Bidirectional)

```
**{octicon}`tag;1em` Document Version:** {bdg-secondary}`0.1.0`  
<br>*>{octicon}`calendar;1em` Last Update:** {bdg-secondary}`2026-04-07`
```

Introduction

Overview

This document defines a **bidirectional** mapping between the **IDTA 02035-1 Digital Battery Passport – Part 1: Digital Nameplate** (DBP Nameplate) and OpenUSD. It is a self-contained reference covering both translation directions:

- **DBP → USD:** How AAS Submodel data is encoded in a USD prim using `DppNameplateAPI` and `BatteryNameplateAPI` applied schemas, including schema definitions, encoding rules, and usage examples.
- **USD → DBP:** How USD prim data is read back and reconstructed into a conformant AAS `BatteryNameplate` Submodel instance, including reconstruction steps, fallback rules, and round-trip fidelity analysis.

The DBP Nameplate is a Submodel Template of the Asset Administration Shell (AAS), defining the static identification and marking attributes required by EU Battery Regulation (EU) 2023/1542 and DIN DKE SPEC 99100. This mapping is also a concrete example supporting the **Identifier Separation of Concerns** proposal: the DBP Nameplate contains multiple distinct identifier types (product URI, serial number, manufacturer ID, facility ID, document references) that the proposal explicitly addresses.

```
**Dependency on the Source Identifier proposal.** Several mappings in this document – in particular `URIOfTheProduct`, `ManufacturerIdentifier`, `EUDeclarationOfConformity`, and `ResultsOfTestReportsProvingCompliance` – use a `sourceId:dpp:*` source identifier mechanism to store external AAS identifiers verbatim alongside USD attributes. This mechanism is defined in the [Identifier Separation of Concerns proposal](./README.md) and is not yet part of the OpenUSD standard. The mappings in this document are written on the assumption that the proposal will be accepted and standardized. Where `assetInfo` sub-dictionaries or `asset[]` attributes are mentioned as alternatives, these are fallback encodings for toolchains that predate source id support.
```

References

DBP Reference

Version	Reference Documents
1.0 (February 2026)	IDTA 02035-1: Digital Battery Passport – Part 1: Digital Nameplate

Version	Reference Documents
—	DIN DKE SPEC 99100: Requirements for data attributes of the battery passport (February 2025)
—	EU Battery Regulation (EU) 2023/1542, Article 77 and Annex XIII
—	IDTA-02006: Digital Nameplate for Industrial Equipment 3.0 (parent template)
—	IDTA-01001-3-1-2: Specification of the Asset Administration Shell Part 1: Metamodel (V3.1.2)

OpenUSD Reference

Version	Reference Documents
24.08	OpenUSD C++ and Schema Documentation , OpenUSD Github Repository , USD Terms and Concepts
— (proposed)	Identifier Separation of Concerns — OpenUSD proposal defining the <code>sourceId:*</code> source identifier mechanism used in this document

General Assumptions and Constraints

- **Two-way mapping** (DBP ↔ USD). Round-trip fidelity is a goal but is not fully achievable; lossy fields are explicitly documented.
- The DBP Nameplate is encoded in USD using two applied API schemas: `DppNameplateAPI` (generic nameplate fields from IDTA-02006) and `BatteryNameplateAPI` (battery-specific additions from IDTA-02035). Schema definitions are provided in [BatteryNameplate \(Submodel\)](#).
- **USD → DBP** reconstruction assumes the USD prim has both schemas applied. A prim with only `DppNameplateAPI` cannot produce a complete `BatteryNameplate` submodel instance; it can produce a partial IDTA-02006 nameplate.
- **AAS structural context** (Asset, AssetAdministrationShell, Submodel containment hierarchy) is not represented in USD. The USD → DBP direction produces only a Submodel instance; the surrounding AAS envelope must be provided by the receiving system.
- **MLP (MultiLanguageProperty)** fields suffer a partial round-trip: USD stores only the primary string value. Language variants stored in `customData` survive, but depend on the USD tool respecting that convention.
- **semanticIds** are preserved in `customData` for provenance. They survive round-trip only if USD tooling does not strip `customData`.
- **USD-native concepts** (composition arcs, time-varying attributes, variant sets, relationships) have no DBP equivalent and are silently dropped on USD → DBP export. See [USD-Native Concepts](#).
- USD `token` values for `dpp:lifeCycleStage` map to ECLASS IRDIs via the lookup table in [Appendix A](#). This lookup is required for a conformant USD → DBP export.
- AAS uses a typed metamodel (Submodel, SubmodelElementCollection, SubmodelElementList, Property, MultiLanguageProperty, File). USD has no direct equivalent; the mapping uses applied API schemas, prim attributes, metadata dictionaries, and child prims as appropriate.

Definitions, Acronyms, Abbreviations

Term or Abbreviation	Description
AAS	Asset Administration Shell – the IDTA/Industrie 4.0 digital twin framework
DBP	Digital Battery Passport (IDTA 02035-1 series)
SMT	Submodel Template – a standardized AAS submodel definition
SM	Submodel – top-level container of related data elements in AAS
SMC	SubmodelElementCollection – an unordered named group of SubmodelElements
SML	SubmodelElementList – an ordered list of SubmodelElements of the same type
Prop	Property – a typed scalar SubmodelElement
MLP	MultiLanguageProperty – a Property with per-language-code string values
IRDI	International Registration Data Identifier – semantic ID format (IEC CDD/ECLASS)
semanticId	AAS mechanism for linking a SubmodelElement to a concept definition
<code>DppNameplateAPI</code>	Applied USD API schema for generic DPP nameplate fields (IDTA-02006 origin)
<code>BatteryNameplateAPI</code>	Applied USD API schema for battery-specific nameplate fields (IDTA-02035 additions)
<code>MarkingAPI</code>	Applied USD API schema for a single marking entry
<code>assetInfo</code>	USD metadata dictionary for asset-level identification metadata
<code>customData</code>	USD metadata dictionary for arbitrary user/domain data on a prim
<code>token</code>	A USD value type for interned strings, used for enumerated values
<code>asset</code>	A USD value type (<code>SdfAssetPath</code>) for file or URI references
<code>kind</code>	A USD metadata string classifying a prim's role in the model hierarchy
<code>sourceId:*</code>	Source identifier attributes — namespaced custom or schema-backed attributes storing external system identifiers verbatim (from the Identifier Separation of Concerns proposal)
Round-trip	A DBP → USD → DBP transformation; data survives if the final DBP instance is equivalent to the original
Lossless	A field that survives round-trip without data loss
Lossy	A field that loses information during round-trip (e.g., language variants in MLP)
Dropped	Data that is discarded and cannot be recovered during round-trip

Concepts

High-Level Concept Mapping

DBP (AAS)	Schema	OpenUSD	Round-trip	Notes
BatteryNameplate (Submodel)	both	Prim with <code>DppNameplateAPI</code> + <code>BatteryNameplateAPI</code>	Partial	AAS envelope (Asset, AAS) not round-tripped; Submodel instance only
URIOfTheProduct	<code>DppNameplateAPI</code>	<code>sourceId:dpp:uriOfTheProduct</code> (string) + <code>dpp:uriOfTheProduct</code> (asset)	Lossless	Source id entry is the primary cross-system identifier; <code>assetInfo["identifier"]</code> written as fallback
ManufacturerName	<code>DppNameplateAPI</code>	<code>dpp:manufacturerName</code> (string)	Lossy	Primary language value only; language tag lost
ContactInformation	<code>DppNameplateAPI</code>	<code>dpp:contact:*</code> attributes or child prim	Lossless	All mandatory fields preserved
SerialNumber	<code>DppNameplateAPI</code>	<code>dpp:serialNumber</code> (string)	Lossless	Direct string mapping
DateOfManufacture	<code>DppNameplateAPI</code>	<code>dpp:dateOfManufacture</code> (string)	Lossless	ISO 8601 string preserved
DateOfPuttingIntoService	<code>DppNameplateAPI</code>	<code>dpp:dateOfPuttingIntoService</code> (string)	Lossless	ISO 8601 string preserved; absent if not authored
UniqueFacilityIdentifier	<code>DppNameplateAPI</code>	<code>dpp:uniqueFacilityIdentifier</code> (string)	Lossless	Location identifier; direct string mapping
Markings (SML)	<code>DppNameplateAPI</code>	Child prims under <code>Markings/</code> scope	Lossy	SML ordering requires <code>Marking_NN</code> naming convention
LifeCycleStage	<code>BatteryNameplateAPI</code>	<code>dpp:lifeCycleStage</code> (token)	Lossless	Token ↔ IRDI via lookup table (Appendix A)
OperatorIdentifier	<code>BatteryNameplateAPI</code>	<code>dpp:operatorIdentifier</code> (string)	Lossless	Optional; absent if not authored
ManufacturerIdentifier	<code>BatteryNameplateAPI</code>	<code>sourceId:dpp:manufacturerIdentifier</code> (string) + <code>dpp:manufacturerIdentifier</code> (string)	Lossless	Source id entry is the primary cross-system identifier; <code>assetInfo["name"]</code> must not be used
EUDeclarationOfConformity	<code>BatteryNameplateAPI</code>	<code>sourceId:dpp:euDeclarationOfConformityIds</code> (string[]) primary; <code>dpp:euDeclarationOfConformity</code> (asset[]) for URI/path cases	Lossless	Source id string array preserves verbatim AAS document identifiers
ResultsOfTestReportsProvingCompliance	<code>BatteryNameplateAPI</code>	<code>sourceId:dpp:testReportComplianceIds</code> (string[]) primary; <code>dpp:testReportCompliance</code> (asset[]) for URI/path cases	Lossless	Same as above
—	—	<code>assetInfo["version"]</code>	Dropped	No DBP equivalent; USD-native asset versioning
—	—	Composition arcs	Dropped	No DBP equivalent
—	—	Time-varying attributes	Dropped	No DBP equivalent
—	—	Variant sets	Dropped	No DBP equivalent
—	—	Relationships	Dropped	No DBP equivalent

BatteryNameplate (Submodel)

The `BatteryNameplate` submodel maps to **two composed applied API schemas** in USD, mirroring the spec's own derivation structure:

- `DppNameplateAPI` — the generic digital product nameplate, encoding the fields inherited from IDTA-02006 (Digital Nameplate for Industrial Equipment 3.0). Applicable to any product with a digital product passport.
- `BatteryNameplateAPI` — the battery-specific extension, encoding the five fields added by IDTA-02035: `LifeCycleStage`, `OperatorIdentifier`, `ManufacturerIdentifier`, `EUDeclarationOfConformity`, and `ResultsOfTestReportsProvingCompliance`.

Both schemas use the `dpp:` namespace prefix. The prim `kind` should be `component`. The AAS `semanticId` of the submodel is preserved in `customData` for provenance.

Schema Definitions (illustrative)

```
# — Generic DPP nameplate (IDTA-02006 origin) —————
class "DppNameplateAPI" (
  inherits = </APISchemaBase>
  doc = "Full IDTA 02006-3-0 Digital Nameplate for Industrial Equipment base schema.
Applicable to any product with a digital nameplate or digital product passport. For battery
passport workflows apply together with BatteryNameplateAPI; for full industrial equipment
workflows apply together with IndustrialEquipmentAPI. See
digital_nameplate_openusd_bidirectional_mapping.md for per-field documentation of fields
marked (IDTA 02006-3-0)."
```

customData = {
 token apiSchemaType = "singleApply"
}

```
) {
  # — Mandatory identification —————
  asset dpp:uriOfTheProduct (
    doc = "Unique URI identifying this product passport instance. (AAS: URIOfTheProduct,
mandatory) Also written to sourceId:dpp:uriOfTheProduct for cross-system discoverability."
  )
  string dpp:manufacturerName (
    doc = "Legally valid manufacturer name. (AAS: ManufacturerName, MLP, mandatory)
Primary language value; language variants in customData."
  )

  # — Optional product classification (IDTA 02006-3-0) —————
  string dpp:manufacturerProductDesignation (
    doc = "Short description or model name of the product. (AAS:
ManufacturerProductDesignation, MLP, optional)"
  )
  string dpp:manufacturerProductRoot (
    doc = "Highest-level product grouping defined by the manufacturer. (AAS:
ManufacturerProductRoot, MLP, optional)"
  )
  string dpp:manufacturerProductFamily (
    doc = "Product family designation. (AAS: ManufacturerProductFamily, MLP, optional)"
  )
  string dpp:manufacturerProductType (
    doc = "Product type or variant designation. (AAS: ManufacturerProductType, MLP,
optional)"
  )
  string dpp:orderCode (
    doc = "Manufacturer order code / catalog number. (AAS: OrderCodeOfManufacturer, MLP,
optional) Also written to sourceId:dpp:orderCode when used as a cross-system catalog key."
  )
  string dpp:articleNumber (
    doc = "Manufacturer product article number. (AAS:
ProductArticleNumberOfManufacturer, MLP, optional) Also written to
sourceId:dpp:articleNumber when used as a PLM/ERP cross-system key."
  )

  # — Instance and batch identifiers —————
  string dpp:serialNumber (
```

```

        doc = "Per-instance serial number. (AAS: SerialNumber, mandatory in battery passport
context)"
    )
    string dpp:batchNumber (
        doc = "Manufacturing batch or lot number. (AAS: BatchNumber, xs:string, optional –
IDTA 02006-3-0)"
    )

    # — Origin, dates, and facility —————
    string dpp:countryOfOrigin (
        doc = "ISO 3166-1 alpha-2 country code of origin. (AAS: CountryOfOrigin, xs:string,
optional – IDTA 02006-3-0)"
    )
    string dpp:yearOfConstruction (
        doc = "Year of manufacture in YYYY format. (AAS: YearOfConstruction, xs:string,
optional – IDTA 02006-3-0)"
    )
    string dpp:dateOfManufacture (
        doc = "Manufacturing date in ISO 8601 format YYYY-MM-DD. (AAS: DateOfManufacture)"
    )
    string dpp:dateOfPuttingIntoService (
        doc = "Service start date in ISO 8601 format YYYY-MM-DD. Optional. (AAS:
DateOfPuttingIntoService)"
    )
    string dpp:uniqueFacilityIdentifier (
        doc = "Unique identifier of the manufacturing facility. (AAS:
UniqueFacilityIdentifier)"
    )

    # — Version information (IDTA 02006-3-0) —————
    string dpp:hardwareVersion (
        doc = "Hardware revision string. (AAS: HardwareVersion, MLP, optional)"
    )
    string dpp:firmwareVersion (
        doc = "Firmware revision string. (AAS: FirmwareVersion, MLP, optional)"
    )
    string dpp:softwareVersion (
        doc = "Software revision string. (AAS: SoftwareVersion, MLP, optional)"
    )

    # — Media (IDTA 02006-3-0) —————
    asset dpp:companyLogo (
        doc = "Manufacturer company logo image. (AAS: CompanyLogo, File, optional)"
    )

    # — Contact information (IDTA 02002-1 drop-in) —————
    # Flat Option 1 encoding. For Option 2 (child prim) see ContactInformation field
section.
    string dpp:contact:street (
        doc = "Street address. (AAS: ContactInformation/Street, MLP, optional)"
    )
    string dpp:contact:zipcode (
        doc = "Postal code. (AAS: ContactInformation/Zipcode, MLP, optional)"
    )

```

```

    )
    string dpp:contact:cityTown (
        doc = "City or town. (AAS: ContactInformation/CityTown, MLP, optional)"
    )
    string dpp:contact:stateCounty (
        doc = "State, county, or province. (AAS: ContactInformation/StateCounty, MLP,
optional)"
    )
    string dpp:contact:nationalCode (
        doc = "ISO 3166-1 alpha-2 country code for contact address. (AAS:
ContactInformation/NationalCode, MLP, optional)"
    )
    string dpp:contact:poBox (
        doc = "Post office box. (AAS: ContactInformation/POBox, MLP, optional)"
    )
    string dpp:contact:company (
        doc = "Company name at the contact address. (AAS: ContactInformation/Company, MLP,
optional)"
    )
    string dpp:contact:department (
        doc = "Department within the company. (AAS: ContactInformation/Department, MLP,
optional)"
    )
    string dpp:contact:timeZone (
        doc = "IANA time zone identifier. (AAS: ContactInformation/TimeZone, xs:string,
optional)"
    )
    string dpp:contact:phone (
        doc = "Primary telephone number. (AAS: ContactInformation/Phone/TelephoneNumber,
MLP, optional)"
    )
    string dpp:contact:fax (
        doc = "Primary fax number. (AAS: ContactInformation/Fax/FaxNumber, MLP, optional)"
    )
    string dpp:contact:email (
        doc = "Primary email address. (AAS: ContactInformation/Email/EmailAddress,
xs:string, optional)"
    )
    asset dpp:contact:additionalLink (
        doc = "Web address or URI. (AAS:
ContactInformation/IPCommunication/AddressOfAdditionalLink, xs:anyURI, optional)"
    )
    string dpp:contact:nameOfContact (
        doc = "Family name of the primary contact. (AAS: ContactInformation/NameOfContact,
MLP, optional)"
    )
    string dpp:contact:firstName (
        doc = "First name of the primary contact. (AAS: ContactInformation/FirstName, MLP,
optional)"
    )
    string dpp:contact:remarks (
        doc = "Additional address remarks. (AAS: ContactInformation/AddressRemarks, MLP,
optional)"

```

```

    )
}

# — Battery-specific extension (IDTA-02035 additions) —————
class "BatteryNameplateAPI" (
    inherits = </APISchemaBase>
    doc = "Battery-specific nameplate extension. Encodes the five fields added by IDTA-02035-1 over the base IDTA-02006 template. Apply together with DppNameplateAPI."
    customData = {
        token apiSchemaType = "singleApply"
    }
) {
    token dpp:lifecycleStage (
        allowedTokens = ["original", "repurposed", "re-used", "remanufactured", "waste"]
        doc = "Life cycle status of the battery per EU Battery Regulation. (AAS: LifecycleStage)"
    )
    string dpp:operatorIdentifier (
        doc = "Unique operator identifier per ISO/IEC 15459. Optional. (AAS: OperatorIdentifier)"
    )
    string dpp:manufacturerIdentifier (
        doc = "Formal identifier of the manufacturer. (AAS: ManufacturerIdentifier) Also written to sourceId:dpp:manufacturerIdentifier for cross-system discoverability."
    )
    asset[] dpp:euDeclarationOfConformity (
        doc = "Asset paths or URIs to EU Declaration of Conformity documents. (AAS: EUDeclarationOfConformity) Written only when document identifiers are resolvable URIs or file paths. For verbatim AAS document identifier strings, use sourceId:dpp:euDeclarationOfConformityIds (string[])."

```



```

    string marking:issueDate (
        doc = "Certificate issue date in ISO 8601 format. Optional. (AAS: IssueDate)"
    )
    string marking:expiryDate (
        doc = "Certificate expiry date in ISO 8601 format. Optional. (AAS: ExpiryDate)"
    )
    asset marking:file (
        doc = "Image file of the conformity symbol. Optional. (AAS: MarkingFile)"
    )
    string[] marking:additionalText (
        doc = "Additional explanatory text, e.g. notified body ID. Optional, multi-value.
(AAS: MarkingAdditionalText)"
    )
}

```

DBP → USD

A `BatteryNameplate` Submodel instance is encoded as a USD prim with both `DppNameplateAPI` and `BatteryNameplateAPI` applied. The AAS `semanticId` is stored in `customData`.

DBP	OpenUSD	Description
Submodel semanticId (IDTA-02006)	<code>customData["aas:submodelSemanticId:nameplate"]</code> (string)	Preserves the IDTA-02006 parent template semantic ID
Submodel semanticId (IDTA-02035-1)	<code>customData["aas:submodelSemanticId:battery"]</code> (string)	Preserves <code>https://admin-shell.io/idta/digitalbatterypassport/nameplate/1/0/Nameplate</code>

Usage Example

```

def xform "Battery_A12345X75EN" (
    kind = "component"
    prepend apiSchemas = ["DppNameplateAPI", "BatteryNameplateAPI"]
    customData = {
        string "aas:submodelSemanticId:nameplate" = "https://admin-shell.io/idta/nameplate/3/0"
        string "aas:submodelSemanticId:battery"   = "https://admin-shell.io/idta/digitalbatterypassport/nameplate/1/0/Nameplate"
        string "aas:semanticId:uriOfTheProduct"    = "0112/2///61987#ABN590#002"
        dictionary "dpp:manufacturerName:i18n" = {
            string "de" = "Muster AG"
            string "en" = "Muster AG"
        }
    }
    assetInfo = {
        # Fallback for USD tools without source id support
        asset identifier = @https://dcqr.com/?m=R123456789@
    }
) {
    # — Source identifier entries (primary – requires source id proposal) —
    custom string    sourceId:dpp:uriOfTheProduct          = "https://dcqr.com/?m=R123456789"
    custom string    sourceId:dpp:manufacturerIdentifier   = "XYZ-123456"
    custom string[]  sourceId:dpp:euDeclarationOfConformityIds = ["urn:example:doc:eu-conformity-2024-001"]
}

```

```

    custom string[] sourceId:dpp:testReportComplianceIds = ["urn:example:doc:test-
report-2024-001"]

# — DppNameplateAPI fields —————
asset dpp:uriOfTheProduct = @https://dcqr.com/?m=R123456789@
string dpp:manufacturerName = "Muster AG"
string dpp:serialNumber = "A12345-X75EN"
string dpp:dateOfManufacture = "2022-01-01"
string dpp:dateOfPuttingIntoService = "2022-06-01"
string dpp:uniqueFacilityIdentifier = "987654321"

# — BatteryNameplateAPI fields —————
token dpp:lifecycleStage = "original"
string dpp:manufacturerIdentifier = "XYZ-123456"
# asset[] written only when document identifiers are resolvable URIs/paths
asset[] dpp:euDeclarationOfConformity = [./docs/eu_doc_conformity_en.pdf@]
asset[] dpp:testReportCompliance = [./docs/test_report_compliance.pdf@]

# — Markings —————
def Scope "Markings" {
    def Scope "Marking_00" (prepend apiSchemas = ["MarkingAPI"]) {
        custom string sourceId:dpp:marking:certificationId = "KEMA99IECEX1105/128"
        token marking:name = "0173-1#07-DAA603#004"
        string marking:issueDate = "2022-01-01"
        string marking:expiryDate = "2028-01-01"
        asset marking:file = @./markings/markings_ce.png@
        string[] marking:additionalText = ["0123"]
    }
    def Scope "Marking_01" (prepend apiSchemas = ["MarkingAPI"]) {
        token marking:name = "WEEE"
        asset marking:file = @./markings/WEEE.png@
    }
}
}

```

USD → DBP

To reconstruct a `BatteryNameplate` Submodel from a USD prim:

- 1. Identify the prim:** The prim must have both `DppNameplateAPI` and `BatteryNameplateAPI` in its applied schemas. A prim with only `DppNameplateAPI` produces an IDTA-02006 nameplate, not a `BatteryNameplate`.
- 2. Construct the Submodel envelope:**
 - `idShort`: `BatteryNameplate`
 - `semanticId`: `https://admin-shell.io/idta/digitalbatterypassport/nameplate/1/0/Nameplate` (from `customData["aas:submodelSemanticId:battery"]` if present; otherwise use the fixed value)
- 3. Populate SubmodelElements** from prim attributes per the per-field mappings below.
- 4. AAS envelope** (Asset, AssetAdministrationShell): cannot be reconstructed from USD alone. The receiving AAS system must supply this.

Round-trip Fidelity

AAS Element	Survives Round-trip?	Notes
Submodel idShort	Yes	Fixed value <code>BatteryNameplate</code>
Submodel semanticId	Yes	Stored in <code>customData</code> ; survives if not stripped
SubmodelElement values	See per-field table	—
Asset, AAS hierarchy	No	Not encoded in USD

URIOfTheProduct

The battery passport URI is the globally unique identifier for this battery passport instance. It is a cross-system identifier (referencing the DPP/AAS ecosystem) and is therefore stored both as a source id entry (primary) and as a typed schema attribute (domain-specific).

DBP → USD

`URIOfTheProduct` is written to three locations, in order of preference:

- `sourceId:dpp:uriofTheProduct` (string) — the primary cross-system identifier entry. Stores the URI verbatim as a plain string, free from `SdfAssetPath` resolution semantics.
- `dpp:uriofTheProduct` (asset) — the domain-specific schema attribute, kept for schema completeness and USD tools that resolve asset paths.
- `assetInfo["identifier"]` — written as a fallback for USD tools that predate source id support. Limited to model-root primis; keys are oriented toward Pixar's asset management workflows and do not carry the semantic specificity of the DPP product URI.

The AAS `semanticId` may be preserved in `customData`:

```
customData = {
  string "aas:semanticId:uriofTheProduct" = "0112/2///61987#ABN590#002"
}
```

Properties

	Name	AAS Type	USD Type	Notes
DBP	<code>URIOfTheProduct</code>	<code>xs:anyURI</code>	—	Mandatory (cardinality 1). AAS semanticId: <code>0112/2///61987#ABN590#002</code>
OpenUSD (primary)	<code>sourceId:dpp:uriofTheProduct</code>	—	<code>string</code>	Verbatim URI; no path-resolution semantics
OpenUSD (schema attr)	<code>dpp:uriofTheProduct</code>	—	<code>asset</code> (<code>SdfAssetPath</code>)	Schema-level discoverability; asset type accommodates URIs
OpenUSD (fallback)	<code>assetInfo["identifier"]</code>	—	<code>asset</code>	Legacy fallback; model roots only

USD → DBP

Read in priority order: `sourceId:dpp:uriOfTheProduct` → `dpp:uriOfTheProduct` → `assetInfo["identifier"]`. Write to AAS Property `URIOfTheProduct` with `valueType = xs:anyURI`.

Round-trip

	DBP → USD	USD → DBP
Value	<code>xs:anyURI</code> → string (source id) + asset (attribute)	string value → <code>xs:anyURI</code>
Fidelity	Lossless	Lossless
Fallback chain	—	<code>sourceId:dpp:uriOfTheProduct</code> → <code>dpp:uriOfTheProduct</code> → <code>assetInfo["identifier"]</code>

ManufacturerName

The legally valid name of the manufacturer. In AAS this is a MultiLanguageProperty (MLP); USD has no native per-locale string type, so the primary string value is stored directly and language variants are preserved in `customData`.

DBP → USD

The primary language string value of the MLP is written to `dpp:manufacturerName`. If language variants are important, they are written to `customData["dpp:manufacturerName:i18n"]` as a dictionary keyed by ISO 639 language code.

Properties

	Name	AAS Type	USD Type	Notes
DBP	<code>ManufacturerName</code>	MLP (MultiLanguageProperty)	—	Mandatory (cardinality 1). AAS semanticId: <code>0112/2///61987#ABA565#009</code>
OpenUSD	<code>dpp:manufacturerName</code>	—	<code>string</code>	Primary language value only

```
string dpp:manufacturerName = "Muster AG"
customData = {
  dictionary "dpp:manufacturerName:i18n" = {
    string "de" = "Muster AG"
    string "en" = "Muster AG"
  }
}
```

USD → DBP

Read `dpp:manufacturerName` (string). Reconstruct as MLP with a single entry. If `customData["dpp:manufacturerName:i18n"]` is present, reconstruct the full MLP from the dictionary.

Round-trip

	DBP → USD	USD → DBP
Primary value	Lossless	Lossless
Language variants	Lossy — written to <code>customData</code> only	Recovered only if <code>customData</code> preserved

ContactInformation

The manufacturer's physical address is structured in AAS as an SMC drop-in from IDTA 02002-1 Contact Information. The mandatory fields per DIN DKE SPEC 99100 are Street, Zipcode, CityTown, NationalCode; optionally Email and AddressOfAdditionalLink.

DBP → USD

Two encoding options exist:

Option 1 — Namespaced attributes (flat, recommended for round-trip): Attributes with the `dpp:contact:` prefix on the same prim. Simple to author; flat list of attributes.

Option 2 — Child prim: A child `Scope` prim (e.g., `Address`) under the battery prim. Cleaner namespace separation; allows the address to be referenced or overridden independently. Recommended when address data is shared across multiple battery instances or updated independently; requires agreement on prim naming for round-trip.

Properties

DBP (ContactInformation)	USD Attribute	AAS Type	USD Type
Street	<code>dpp:contact:street</code>	MLP	<code>string</code>
Zipcode	<code>dpp:contact:zipcode</code>	MLP	<code>string</code>
CityTown	<code>dpp:contact:cityTown</code>	MLP	<code>string</code>
NationalCode	<code>dpp:contact:nationalCode</code>	MLP	<code>string</code>
Email	<code>dpp:contact:email</code>	string (optional)	<code>string</code>
AddressOfAdditionalLink	<code>dpp:contact:additionalLink</code>	<code>xs:anyURI</code> (optional)	<code>asset</code>

Usage Example — Option 1 (namespaced attributes)

```
def xform "Battery_A12345X75EN" (  
  prepend apiSchemas = ["DppNameplateAPI", "BatteryNameplateAPI"]  
) {  
  string dpp:contact:street      = "Sample Street 1"  
  string dpp:contact:zipcode     = "12345"  
  string dpp:contact:cityTown   = "City"  
  string dpp:contact:nationalCode = "DE"  
  string dpp:contact:email      = "contact@muster-ag.de"  
  asset dpp:contact:additionalLink = @https://www.muster-ag.de@  
}
```

Usage Example — Option 2 (child prim)

```
def xform "Battery_A12345X75EN" (  
  prepend apiSchemas = ["DppNameplateAPI", "BatteryNameplateAPI"]  
) {  
  def Scope "Address" {  
    string street      = "Sample Street 1"  
    string zipcode     = "12345"  
    string cityTown    = "City"  
    string nationalCode = "DE"  
    string email       = "contact@muster-ag.de"  
    asset additionalLink = @https://www.muster-ag.de@  
  }  
}
```

USD → DBP

Read `dpp:contact:*` attributes (Option 1) or the child prim named `Address` (Option 2). Reconstruct as SMC `ContactInformation` using the drop-in from IDTA 02002-1. Map each attribute to the corresponding MLP SubmodelElement.

Round-trip

DBP Field	USD Attribute	Fidelity
Street	<code>dpp:contact:street</code>	Lossless
Zipcode	<code>dpp:contact:zipcode</code>	Lossless
CityTown	<code>dpp:contact:cityTown</code>	Lossless
NationalCode	<code>dpp:contact:nationalCode</code>	Lossless
Email	<code>dpp:contact:email</code>	Lossless
AddressOfAdditionalLink	<code>dpp:contact:additionalLink</code>	Lossless
Language variants (MLP)	<code>customData</code>	Lossy — same as ManufacturerName

SerialNumber

The per-instance serial number uniquely identifies an individual battery unit.

DBP → USD

Written to `dpp:serialNumber` (string).

Properties

	Name	AAS Type	USD Type	Notes
DBP	<code>SerialNumber</code>	<code>xs:string</code>	—	Mandatory (cardinality 1). AAS semanticId: <code>0112/2///61987#ABA951#009</code>
OpenUSD	<code>dpp:serialNumber</code>	—	<code>string</code>	Instance-level identifier

USD → DBP

Read `dpp:serialNumber`. Write to AAS `Property SerialNumber` with `valueType = xs:string`.

Round-trip: Lossless

DateOfManufacture

Manufacturing date of the individual battery item. Per DBP, this must be per-instance (not per-model). ISO 8601 format (YYYY-MM-DD). USD has no native `date` type; ISO 8601 strings stored as `string` attributes are used.

DBP → USD

Written to `dpp:dateOfManufacture` as ISO 8601 string `YYYY-MM-DD`.

Properties

	Name	AAS Type	USD Type	Notes
DBP	<code>DateOfManufacture</code>	<code>xs:date</code>	—	Mandatory (cardinality 1). AAS semanticId: <code>0112/2///61987#ABB757#007</code> . Must comply with ISO 8601-1:2020.
OpenUSD	<code>dpp:dateOfManufacture</code>	—	<code>string</code>	ISO 8601 date string convention

USD → DBP

Read `dpp:dateOfManufacture`. Validate ISO 8601-1:2020 format. Write to AAS `Property DateOfManufacture` with `valueType = xs:date`.

Round-trip: Lossless (provided the string is valid ISO 8601)

DateOfPuttingIntoService

Optional service start date (cardinality 0..1). Same date string convention as DateOfManufacture.

DBP → USD

Written to dpp:dateOfPuttingIntoService as ISO 8601 string if present. If absent in DBP, the attribute is not authored in USD.

USD → DBP

If dpp:dateOfPuttingIntoService is authored and non-empty, write to AAS Property DateOfPuttingIntoService with valueType = xs:date. If absent or empty, omit the optional SubmodelElement.

Round-trip: Lossless

UniqueFacilityIdentifier

A unique string identifying the manufacturing location. This is a location identifier, distinct from the product identifier (URIOfTheProduct) and the manufacturer identifier (ManufacturerIdentifier). The Identifier Separation of Concerns proposal explicitly categorizes these as separate concerns.

DBP → USD

Written to dpp:uniqueFacilityIdentifier (string).

Properties

	Name	AAS Type	USD Type	Notes
DBP	UniqueFacilityIdentifier	xs:string	—	Mandatory (cardinality 1). AAS semanticId: https://admin-shell.io/idta/nameplate/3/0/UniqueFacilityIdentifier
OpenUSD	dpp:uniqueFacilityIdentifier	—	string	Location/facility scope identifier

USD → DBP

Read dpp:uniqueFacilityIdentifier. Write to AAS Property UniqueFacilityIdentifier with valueType = xs:string.

Round-trip: Lossless

Markings

The Markings SML contains one or more Marking SMCs, each representing a regulatory or conformity symbol on the physical battery label (e.g., CE marking, WEEE symbol, carbon footprint label). Each marking has a name, optional certificate designation, optional validity dates, an optional image file, and optional additional text.

In USD, each marking is represented as a child prim under a `Markings` scope, with `MarkingAPI` applied. Child prims are used because:

- Markings have heterogeneous optional fields
- Marking image files (`MarkingFile`) are naturally `asset`-typed attributes
- Multiple markings need to be independently addressable

Composition

```
Battery_A12345X75EN (Xform, DppNameplateAPI + BatteryNameplateAPI applied)
└─ Markings (Scope)
    └─ Marking_00 (Scope, MarkingAPI applied)
    └─ Marking_01 (Scope, MarkingAPI applied)
```

DBP → USD

Each `Markings__NN__` SMC is written as a child prim under a `Markings` Scope, with `MarkingAPI` applied. Prim names are constructed as `Marking_NN` (zero-padded index) to preserve the original SML ordering.

Properties (per Marking)

DBP (Markings00)	USD Attribute	AAS Type	USD Type
MarkingName	<code>marking:name</code>	<code>xs:string</code> (IRDI preferred)	<code>token</code>
DesignationOfCertificateOrApproval	<code>sourceId:dpp:marking:certificationId</code>	<code>xs:string</code>	<code>string</code>
IssueDate	<code>marking:issueDate</code>	<code>xs:date</code>	<code>string</code> (ISO 8601)
ExpiryDate	<code>marking:expiryDate</code>	<code>xs:date</code>	<code>string</code> (ISO 8601)
MarkingFile	<code>marking:file</code>	<code>File</code>	<code>asset</code>
MarkingAdditionalText	<code>marking:additionalText</code>	<code>xs:string[]</code>	<code>string[]</code>

`sourceId:dpp:marking:certificationId` is the sole encoding for the certificate or approval number. Using a source id rather than a schema attribute avoids duplication and enables cross-system linking to approval databases (ATEX, IECEx, etc.) without requiring `MarkingAPI` schema knowledge. Omitted when no certificate designation is present.

`MarkingName` preferred value is an IRDI from IEC CDD/ECLASS (e.g., CE = `0173-1#07-DAA603#004`). Free-text names are stored as `token` values but may be subject to token normalisation.

Usage Example

```
def Scope "Markings" {
  def Scope "Marking_00" (prepend apiSchemas = ["MarkingAPI"]) {
    custom string sourceId:dpp:marking:certificationId = "KEMA99IECEX1105/128"
    token      marking:name                        = "0173-1#07-DAA603#004"
    string     marking:issueDate                    = "2022-01-01"
    string     marking:expiryDate                   = "2028-01-01"
    asset      marking:file                         = @./markings/markings_ce.png@
    string[]   marking:additionalText                = ["0123"]
  }
  def Scope "Marking_01" (prepend apiSchemas = ["MarkingAPI"]) {
```

```

    token marking:name = "WEEE"
    asset marking:file = @./markings/WEEE.png@
  }
}

```

USD → DBP

1. Find the child prim named `Markings` (Scope).
2. Collect all child prims that have `MarkingAPI` applied.
3. **Sort by prim name** to recover the original SML order (relies on the `Marking_NN` naming convention).
4. For each, construct a `Markings__NN__` SMC with the fields above.

Round-trip

DBP Field	USD Attribute	Fidelity
MarkingName	<code>marking:name</code> (token)	Lossless if IRDI token; lossy if free-text token normalised
DesignationOfCertificateOrApproval	<code>sourceId:dpp:marking:certificationId</code> (string)	Lossless
IssueDate	<code>marking:issueDate</code> (string)	Lossless
ExpiryDate	<code>marking:expiryDate</code> (string)	Lossless
MarkingFile	<code>marking:file</code> (asset)	Lossless (path preserved)
MarkingAdditionalText	<code>marking:additionalText</code> (string[])	Lossless
SML ordering	Child prim sort by <code>Marking_NN</code> name	Lossless if naming convention respected; otherwise ordering lost

LifeCycleStage

Schema: `BatteryNameplateAPI`

Indicates the current lifecycle state of the battery. The DBP defines five enumerated values derived from ECLASS IRDIs. In USD, `token` type with `allowedTokens` metadata is the natural fit for enumerated values.

DBP → USD

Map the ECLASS IRDI value to the corresponding USD token using the table below. Write to `dpp:lifecycleStage`.

Properties

	Name	AAS Type	USD Type	Notes
DBP	LifecycleStage	xs:string (ECLASS IRDI enum)	—	Mandatory (cardinality 1)
OpenUSD	dpp:lifecycleStage	—	token	allowedTokens: "original", "repurposed", "re-used", "remanufactured", "waste"

ECLASS IRDI (DBP value)	USD Token
0173-1#07-ACC020#001	original
0173-1#07-ACC021#001	repurposed
0173-1#07-ACC022#001	re-used
0173-1#07-ACC023#001	remanufactured
0173-1#07-ACC024#001	waste

See also [Appendix A](#) for the authoritative lookup table used by USD → DBP export.

USD → DBP

Read `dpp:lifecycleStage` (token). Reverse-map to the ECLASS IRDI using [Appendix A](#). Write to AAS `Property LifecycleStage` with `valueType = xs:string` and the IRDI as the value.

Round-trip: Lossless (via lookup table)

OperatorIdentifier

Schema: `BatteryNameplateAPI`

Optional identifier of the battery operator per ISO/IEC 15459 series (cardinality 0..1).

DBP → USD

Written to `dpp:operatorIdentifier` (string) if present. If absent in DBP, the attribute is not authored.

Properties

	Name	AAS Type	USD Type	Notes
DBP	OperatorIdentifier	xs:string	—	Optional (cardinality 0..1)
OpenUSD	dpp:operatorIdentifier	—	string	Not authored if absent in DBP

USD → DBP

If `dpp:operatorIdentifier` is authored and non-empty, write to AAS `Property operatorIdentifier`. If absent, omit the optional `SubmodelElement`.

Round-trip: Lossless

ManufacturerIdentifier

Schema: `BatteryNameplateAPI`

A formal identifier for the manufacturer (distinct from the human-readable `ManufacturerName`). This is a **manufacturer-scoped cross-system identifier** and is therefore stored both as a source id entry (primary, for cross-system discoverability) and as a typed schema attribute (domain-specific).

DBP → USD

Written to two locations:

- 1. `sourceId:dpp:manufacturerIdentifier` (string) — the primary cross-system identifier entry, making the formal manufacturer identifier discoverable without prior knowledge of the `dpp:` schema.
- 2. `dpp:manufacturerIdentifier` (string) — the domain-specific schema attribute, kept for schema completeness.

Do **not** mirror to `assetInfo["name"]`. That field carries display-name semantics and is on a deprecation path following the UI Hints migration (`displayName` moved to `uiHints` dictionary). Using it to carry a formal manufacturer identifier conflates identification with presentation. Older content that used `assetInfo["name"]` as a workaround should be migrated to the source id entry.

Properties

	Name	AAS Type	USD Type	Notes
DBP	<code>ManufacturerIdentifier</code>	<code>xs:string</code>	—	Mandatory (cardinality 1). AAS semanticId: <code>urn:samm:io.adminshell.idta.batterypass.technical_data:1.0.0#ManufacturerIdentifier</code>
OpenUSD (primary)	<code>sourceId:dpp:manufacturerIdentifier</code>	—	<code>string</code>	Cross-system discoverable identifier
OpenUSD (schema attr)	<code>dpp:manufacturerIdentifier</code>	—	<code>string</code>	Schema-namespaced; preserves semantic separation from <code>ManufacturerName</code>

USD → DBP

Read `dpp:manufacturerIdentifier` as the authoritative value. If absent, read `sourceId:dpp:manufacturerIdentifier`. Write to AAS `Property ManufacturerIdentifier` with `valueType = xs:string`.

Round-trip

	DBP → USD	USD → DBP
Value	Direct string	Direct string

	DBP → USD	USD → DBP
Fidelity	Lossless	Lossless
Fallback	—	<code>sourceId:dpp:manufacturerIdentifier</code> if <code>dpp:manufacturerIdentifier</code> absent

EUDeclarationOfConformity

Schema: `BatteryNameplateAPI`

A battery passport must include the EU Declaration of Conformity. In AAS, this is a `SubmodelElementList` of document identifier strings referencing documents in the Handover Documentation submodel (IDTA-02035-2). These identifiers are opaque strings — they may be internal AAS document reference keys, not necessarily URIs or file paths.

DBP → USD

Write to two locations:

1. `sourceId:dpp:euDeclarationOfConformityIds` (`string[]`) — the primary encoding. Stores verbatim AAS document identifier strings, free from `SdfAssetPath` path-resolution semantics.
2. `dpp:euDeclarationOfConformity` (`asset[]`) — written additionally only when the identifiers are resolvable URIs or file paths, so that USD tools can follow the asset reference. If the identifiers are opaque keys, this attribute is omitted or left empty to avoid false path resolution.

Properties

	Name	AAS Type	USD Type	Notes
DBP	<code>[00] DocumentIdentifier</code>	<code>xs:string</code>	—	AAS semanticId: <code>urn:samm:io.adminshell.idta.batterypass.digital_nameplate:1.0.0#euDeclarationOfConformity</code>
OpenUSD (primary)	<code>sourceId:dpp:euDeclarationOfConformityIds</code>	—	<code>string[]</code>	Verbatim AAS document identifier strings
OpenUSD (URI/path cases)	<code>dpp:euDeclarationOfConformity</code>	—	<code>asset[]</code>	Written only when identifiers are resolvable URIs/paths

USD → DBP

Read `sourceId:dpp:euDeclarationOfConformityIds` (`string[]`) as the authoritative source. If absent, fall back to `dpp:euDeclarationOfConformity` (`asset[]`) and extract the string value of each `SdfAssetPath`. Write to AAS SML `EUDeclarationOfConformity` as a list of `Property DocumentIdentifier` elements.

Round-trip

	Fidelity	Notes
Document identifier values	Lossless	Verbatim strings preserved via <code>sourceId:dpp:euDeclarationOfConformityIds</code>
Document identifier values (fallback path)	Lossy	If only <code>asset[]</code> is present, opaque AAS keys may be mangled by <code>SdfAssetPath</code> resolution
List length	Lossless	Array length preserved

ResultsOfTestReportsProvingCompliance

Schema: BatteryNameplateAPI

A battery passport must include test report results proving regulatory compliance. Identical encoding and round-trip behaviour to [EUDeclarationOfConformity](#).

Properties

	Name	AAS Type	USD Type	Notes
DBP	[00] DocumentIdentifier	xs:string	—	AAS semanticId: urn:samm:io.adminshell.idta.batterypass.digital_nameplate:1.0.0#resultsOfTestReportsProvingCompliance
OpenUSD (primary)	sourceId:dpp:testReportComplianceIds	—	string[]	Verbatim AAS document identifier strings
OpenUSD (URL/path cases)	dpp:testReportCompliance	—	asset[]	Written only when identifiers are resolvable URLs/paths

Primary: sourceId:dpp:testReportComplianceIds (string[]); fallback: dpp:testReportCompliance (asset[]) for resolvable URI/path cases.

USD-Native Concepts

These concepts exist in USD but have no equivalent in the DBP/AAS metamodel. They are dropped when exporting from USD to DBP.

Composition Arcs

USD supports references, payloads, inherits, specializes, and over composition arcs. These define how layer stacks are assembled into a scene. DBP has no equivalent; AAS Submodels are monolithic documents.

On USD → DBP export: Flatten the composed prim (resolve all composition arcs) before reading attribute values. The flattened values are exported; the composition structure is dropped.

Time-Varying Attributes

USD attributes can carry time samples (values at specific time codes). DBP properties are static.

On USD → DBP export: Use the default value (UsdAttribute::Get() with UsdTimeCode::Default()). If no default value is authored, use the value at UsdTimeCode(0) . If neither is present, the field is treated as unset.

Variant Sets

USD variant sets allow a prim to carry multiple alternative representations selectable at runtime. DBP has no equivalent.

On USD → DBP export: Export the currently selected variant only. Other variants are dropped.

Relationships

USD relationships are typed connections between prims. DBP uses ReferenceElement for inter-submodel links, but the DBP Nameplate submodel contains no relationship-type SubmodelElements.

On USD → DBP export: Dropped. If a USD relationship encodes information that should map to a DBP field, it must be modelled as an attribute instead.

Appendices

Appendix A: LifeCycleStage IRDI ↔ Token Mapping

This table is required for lossless round-trip of `LifeCycleStage`.

ECLASS IRDI (DBP value)	USD Token (<code>dpp:lifeCycleStage</code>)
0173-1#07-ACC020#001	original
0173-1#07-ACC021#001	repurposed
0173-1#07-ACC022#001	re-used
0173-1#07-ACC023#001	remanufactured
0173-1#07-ACC024#001	waste

DBP → USD: look up IRDI, write token.
USD → DBP: look up token, write IRDI. If the token value is not in this table (e.g., a free-text value), the USD → DBP export must fail validation or write a best-effort string with a warning.

Appendix B: Round-trip Fidelity Summary

DBP Field	Direction	Fidelity	Loss / Condition
URIOfTheProduct	Both	Lossless	Primary: <code>sourceId:dpp:uriOfTheProduct</code> ; fallback chain: <code>dpp:uriOfTheProduct</code> → <code>assetInfo["identifier"]</code>
ManufacturerName (primary value)	Both	Lossless	—
ManufacturerName (language variants)	Both	Lossy	Preserved only via <code>customData</code> convention
ContactInformation (all fields)	Both	Lossless	—
SerialNumber	Both	Lossless	—
DateOfManufacture	Both	Lossless	String must be valid ISO 8601
DateOfPuttingIntoService	Both	Lossless	Absent field handled correctly in both directions
UniqueFacilityIdentifier	Both	Lossless	—
Markings (field values)	Both	Lossless	—
Markings (SML ordering)	Both	Lossless	Requires <code>Marking_NN</code> naming convention
LifeCycleStage	Both	Lossless	Requires IRDI ↔ token lookup table (Appendix A)
OperatorIdentifier	Both	Lossless	Absent field handled correctly in both directions
ManufacturerIdentifier	Both	Lossless	Primary: <code>sourceId:dpp:manufacturerIdentifier</code> ; <code>assetInfo["name"]</code> must not be used
EUDeclarationOfConformity	Both	Lossless	Verbatim identifiers via <code>sourceId:dpp:eUDeclarationOfConformityIds</code> ; fallback <code>asset[]</code> is lossy for opaque AAS keys
ResultsOfTestReportsProvingCompliance	Both	Lossless	Verbatim identifiers via <code>sourceId:dpp:testReportComplianceIds</code> ; same fallback caveat
semanticIds	Both	Conditional	Survive only if <code>customData</code> is not stripped by USD tooling
AAS Asset / AAS hierarchy	DBP → USD	Dropped	Not encoded in USD

DBP Field	Direction	Fidelity	Loss / Condition
Composition arcs	USD → DBP	Dropped	Flatten before export
Time-varying attributes	USD → DBP	Dropped	Use default value
Variant sets	USD → DBP	Dropped	Export selected variant only
Relationships	USD → DBP	Dropped	No DBP Nameplate equivalent

Appendix C: AAS-to-USD and USD-to-AAS Type Mapping

AAS Type	AAS Description	USD Type	DBP → USD	USD → DBP	Round-trip Fidelity
<code>xs:string</code>	Text string	<code>string</code>	Direct	Direct	Lossless
<code>xs:anyURI</code>	Universal Resource Identifier	<code>string</code> (source id) + <code>asset</code> (attr)	URI as source id string + asset path	Source id string → URI	Lossless
<code>xs:date</code>	ISO 8601 date	<code>string</code>	ISO 8601 string	Parse ISO 8601 string to <code>xs:date</code>	Lossless if valid ISO 8601
<code>MLP</code> (primary value)	Per-language-code string map	<code>string</code>	Primary string	Single-entry MLP	Lossless for primary value
<code>MLP</code> (all variants)	Per-language-code string map	<code>string</code> + <code>customData</code> dict	Primary + i18n dict	Reconstruct from dict if present	Lossy if <code>customData</code> stripped
<code>File</code>	Binary file with MIME type	<code>asset</code>	File path as asset	Asset path as file path	Lossless
ECLASS IRDI (enum)	IRDI-coded enumeration value	<code>token</code>	IRDI → token via lookup	Token → IRDI via lookup	Lossless with lookup table
<code>SMC</code> (flat fields)	Named group of elements	Namespaced attributes	<code>ns:field</code> attributes	Read <code>ns:field</code> attributes	Lossless
<code>SML</code> (ordered, complex elements)	Ordered list of same-typed elements	Child prims with index-based names	<code>Scope_NN</code> child prims	Sort child prims by name	Lossless if naming convention respected
<code>SML</code> (ordered, scalar list)	Ordered list of same-typed elements	Array attribute	<code>type[]</code> attribute	Array to list of Properties	Lossless

AAS Type	AAS Description	USD Type	DBP → USD	USD → DBP	Round-trip Fidelity
Opaque string identifier	External document or system key	<code>string[]</code> (source id)	Verbatim string array	Read string array	Lossless

Appendix D: Identifier Encoding Strategies

The DBP Nameplate contains several identifier fields with distinct scopes. The table below summarises which require source id entries and which are sufficiently covered by plain `dpp:*` schema attributes.

Field	Identifier Scope	Primary USD Encoding	Fallback / Legacy
<code>URIOfTheProduct</code>	Per-instance product passport URI	<code>sourceId:dpp:uriOfTheProduct</code> (string)	<code>dpp:uriOfTheProduct</code> (asset) → <code>assetInfo["identifier"]</code>
<code>SerialNumber</code>	Per-instance serial number	<code>dpp:serialNumber</code> (string)	—
<code>ManufacturerIdentifier</code>	Manufacturer registry	<code>sourceId:dpp:manufacturerIdentifier</code> (string)	<code>dpp:manufacturerIdentifier</code> (string)
<code>UniqueFacilityIdentifier</code>	Manufacturing location	<code>dpp:uniqueFacilityIdentifier</code> (string)	—
<code>OperatorIdentifier</code>	Operator scope	<code>dpp:operatorIdentifier</code> (string)	—
<code>EUDeclarationOfConformity</code> document IDs	External document management system	<code>sourceId:dpp:euDeclarationOfConformityIds</code> (string[])	<code>dpp:euDeclarationOfConformity</code> (asset[])
<code>ResultsOfTestReportsProvingCompliance</code> document IDs	External document management system	<code>sourceId:dpp:testReportComplianceIds</code> (string[])	<code>dpp:testReportCompliance</code> (asset[])

Rationale for source id entries. Fields that reference identifiers in external registries or document management systems (`URIOfTheProduct`, `ManufacturerIdentifier`, EU DoC and test report document IDs) benefit from source id entries because:

- They are opaque from USD's perspective — USD cannot validate or resolve them without external context.
- They must survive round-trip verbatim, without the path-normalisation or resolution semantics that `asset` (`SdfAssetPath`) imposes.
- They enable cross-system linking: a USD traversal tool can discover all prims referencing a given manufacturer or document without loading the `dpp` schema.

Fields that do not require source id entries. `SerialNumber`, `UniqueFacilityIdentifier`, and `OperatorIdentifier` are plain strings specific to this prim's asset. They do not require cross-system discoverability through the source id mechanism; `dpp:*` schema attributes are the sufficient and authoritative encoding. If any of these appear in an external PLM or ERP system and cross-system linking is required, a `sourceId:dpp:<field>` entry may be added as a pipeline extension.

Alternative: `assetInfo` sub-dictionary (Approach A)

Under Approach A of the Identifier Separation of Concerns proposal, source identifiers are stored as sub-dictionaries within `assetInfo` rather than as `sourceId:*` custom attributes. For this mapping that would look like:

```

assetInfo = {
  dictionary sourceIdentifiers = {
    dictionary dpp = {
      string uriOfTheProduct          = "https://dcqr.com/?m=R123456789"
      string manufacturerIdentifier    = "XYZ-123456"
      string[] euDeclarationOfConformityIds = ["urn:example:doc:eu-conformity-2024-001"]
      string[] testReportComplianceIds   = ["urn:example:doc:test-report-2024-001"]
    }
  }
}

```

This encoding is equivalent in semantics to the `sourceId:dpp:*` approach and may be preferred in toolchains already using `assetInfo` conventions. The `sourceId:dpp:*` notation used throughout this document corresponds to Approach B/C of the proposal (applied schema or namespaced custom attributes); both are valid encodings pending the proposal's final resolution.